

TermsGuard AI

Reads the small print and flags the parts that are unfair to you, in plain English.

Taiwo Alabi · MSc Artificial Intelligence, National College of Ireland

Live demo: <https://newtazer-terms-guard-ai.hf.space> **Source:** <https://github.com/iamtamilore/terms-guard-ai>

Contents

1. The problem
 2. Results
 3. How it works
 4. What it compares against
 5. Why similarity alone is not enough
 6. The verification layer
 7. Saying how sure it is
 8. Honest limitations
 9. Run it
-

1. The problem

Reading every privacy policy an average internet user meets would take roughly **244 hours a year**. Nobody does it. So people agree to data selling and tracking without knowing they have.

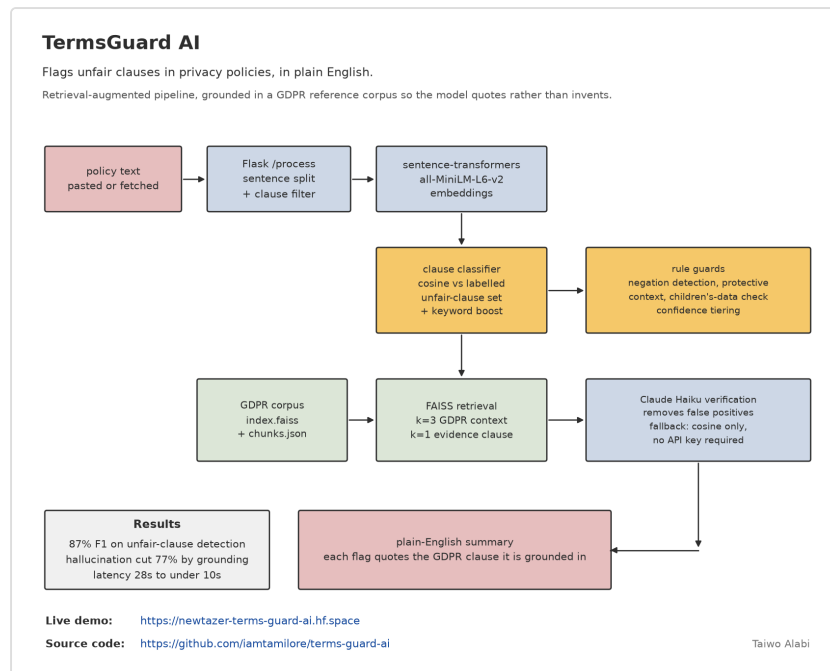
The problem is not that the documents are secret. They are published, free, and legally binding. They are simply unreadable at the length and density they are written, and the one clause that matters is buried among forty that do not.

2. Results

Measure	Result
Unfair-clause detection	87% F1
Invented content, after grounding	cut by 77%
End-to-end latency	28 seconds to under 10
GDPR reference chunks indexed	4,527
Labelled reference clauses	205

The 77% figure is the one I care about most. A tool that tells you a clause is unfair, and is sometimes making it up, is worse than no tool, because it teaches you to distrust the warnings. Grounding each flag in a retrieved GDPR article is what makes the output checkable: every concern points at the legal text it came from.

3. How it works



Retrieval-augmented pipeline. Similarity proposes, rules and retrieval constrain, the LLM verifies.

Text is split into individual clauses and de-duplicated. Each clause is embedded with `all-MiniLM-L6-v2` and compared by cosine similarity against labelled examples of fair and unfair clauses. Anything that looks unfair then has GDPR context retrieved for it from a FAISS index, and Claude Haiku verifies the flag before it is shown.

4. What it compares against

The system carries **205 hand-labelled reference clauses**: 105 examples of fair terms and 100 examples of unfair ones. A new clause is judged by how close it sits to each set.

Separately, a FAISS index holds **4,527 chunks of GDPR text**. When a clause is flagged, retrieval pulls the most relevant articles, so the explanation cites the regulation rather than the model's impression of it.

That split matters. The labelled clauses decide *whether* something is a problem. The GDPR corpus supplies *why*, and it is the second half that makes a flag worth acting on.

5. Why similarity alone is not enough

This is the part I would defend hardest in an interview.

Consider two sentences:

“We sell your personal data to third parties.” “We do **not** sell your personal data to third parties.”

To a sentence embedding these are nearly identical. They share almost every word, the same grammar, the same subject matter. Cosine similarity will place them side by side, and a purely semantic classifier will flag the second sentence as a violation when it is actually a reassurance.

So the pipeline carries a layer of explicit rule guards on top of the similarity score:

- **Negation detection**, for exactly the case above.
- **Protective-context detection**, for clauses that describe a right being granted rather than removed.

- **A children's-data check**, because clauses about minors carry obligations that generic similarity scoring does not weight correctly.
- **Keyword boosting**, for terms whose presence changes the reading regardless of how the sentence embeds.

None of these are clever. They exist because I ran the similarity-only version, read what it got wrong, and found the errors clustered around cases where meaning is inverted by a single word.

6. The verification layer

When an API key is configured, Claude Haiku re-reads each flagged clause and removes false positives before anything is shown to the user.

The design decision worth noting is the **fallback**: with no key set, the tool runs in matching-only mode rather than failing. It degrades to a less precise version of itself instead of refusing to work, which means the demo is always live and the tool is not hostage to a paid dependency.

7. Saying how sure it is

Flags are tiered by confidence rather than presented as flat verdicts, with thresholds at 0.55 for high and 0.45 for medium.

A document-level risk assessment then combines the proportion of clauses flagged with the average confidence across them, so a policy with two borderline flags is not reported the same way as one with fifteen strong ones.

This is the same instinct that runs through my other projects: state uncertainty rather than hide it behind a single number.

8. Honest limitations

- **It is not legal advice**, and the interface says so. It surfaces clauses worth reading properly; it does not tell you your rights.
 - **Grounded in GDPR only**. A policy written for another jurisdiction gets judged against the wrong regulation, and the tool does not currently detect that mismatch.
 - **The 87% F1 is measured on my own labelled set**. 205 reference clauses is a working corpus, not a benchmark, and performance on unusual drafting styles is unmeasured.
 - **This was a contributed project**, not a solo build. My work was the retrieval pipeline, the grounding, the rule guards and the latency reduction.
-

9. Run it

Easiest is the live demo: <https://newtazer-terms-guard-ai.hf.space>

Locally:

```
pip install -r requirements.txt
python server.py # http://127.0.0.1:7860
```

Optional, for the verification layer:

```
export ANTHROPIC_API_KEY="sk-ant-..."
```

Stack: Python · Flask · sentence-transformers (MiniLM) · FAISS · Anthropic API · Docker